study-note-organizer-codex-01 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: 하루치 수업/회의 원문을 붙여넣으면 OpenRouter 기본 모델 2개가 병렬로 "학습 목표 / 학습 내용" 형식의 학습 일지를 만들고, 통합 모델 1개가 두 결과를 합쳐 최종본과 TXT 파일({YYMMDD}_{키워드1}_{키워드2}.txt)을 생성하는 브라우저 전용 웹툴.
- 목적/대상 사용자: IT 교육 과정 수강생 또는 강사가 매일 작성하는 일일 학습 일지 정리 자동화. 개인용 로컬 도구 성격.

영역	사용 기술
프론트엔드	React 19.2, TypeScript 6.0 (strict), Vite 8, TailwindCSS v4 (@tailwindcss/vite), lucide-react
백엔드	없음 (브라우저에서 OpenRouter API 직접 호출)
DB	없음 (localStorage: 설정, 히스토리 50건, 초안)
인프라	미정 (정적 호스팅 가정, 배포 설정 없음)
AI	OpenRouter Chat Completions, 13개 모델 하드코딩 (src/utils/constants.ts:3-17)

- 현재 완성도 추정: 약 85%
  - 근거: PRD 2장 디렉토리 구조와 3장 타입, 4장 프롬프트, 7장 처리 흐름, 10장 다운로드 규격, 11장 저장 키가 거의 그대로 구현되어 있고 tsc -p tsconfig.app.json 이 오류 없이 통과한다. 미구현/부분 구현은 무료 모델 툴팁, 모델 드롭다운 그룹화, 메인 화면 상단 무료 모델 경고 배너(설정 패널 안에만 있음), 데스크톱 3단 레이아웃, README 정리, 테스트 정도다.
- 핵심 기능 목록
  - 날짜 + 원문 입력(최대 5000자, 글자 수 표시, 자동 높이, Ctrl+Enter)
  - 기본 모델 A/B Promise.all 병렬 호출 후 둘 다 성공 시 통합 모델 호출
  - 카드별 모델명/공급사 배지/응답 시간/토큰 수/상태 표시, 복사/TXT 다운로드/다시 요청
  - 통합본에서 [핵심 키워드] 추출 후 파일명 생성 및 본문에서 제거
  - 설정(API Key, 모델 3종, 테마), 히스토리, 입력 초안 localStorage 저장
  - 다크/라이트 모드(시스템 선호 감지)

2. 구조 분석

경로	역할
src/main.tsx	엔트리, AppProvider 로 전역 상태 주입
src/App.tsx (133줄)	파이프라인 오케스트레이션(runAll, runMerge, retryBase, retryFinal), 복사/다운로드, 토스트 렌더
src/context/AppContext.tsx (163줄)	useReducer 기반 전역 상태, localStorage 동기화 effect, 히스토리 아이템 생성
src/hooks/useOpenRouter.ts	fetch 호출, HTTP 상태별 한국어 에러 매핑, latency/usage 측정
src/hooks/useTheme.ts	html.dark 클래스 토글
src/hooks/useLocalStorage.ts	미사용 (어디서도 import 하지 않음)
src/components/*	Header, InputSection, ResultSection, ResultCard, SettingsPanel, HistoryPanel, LoadingSpinner, ErrorDisplay
src/utils/prompts.ts	기본/통합 프롬프트 템플릿 (PRD 4장과 동일)
src/utils/formatters.ts	날짜 분해, 키워드 추출/제거, 파일명 생성

경로	역할
src/utils/constants.ts	모델 목록, 기본 설정, 저장 키, 배지 색상
PRD.md	코드 생성용 프롬프트 형태의 상세 명세
README.md / README copy.md	README.md 는 제목 1줄뿐, 실제 문서는 README copy.md 에 있음
dist/	5/29 빌드 산출물 (프로젝트 내부 git 기준 미추적 상태)

데이터 흐름: 사용자가 InputSection 에서 입력하면 reducer 의 inputText/inputDate 가 갱신되고 effect 가 즉시 draft 키에 저장한다. [정리하기] 클릭 시 App.runAll 이 두 카드를 loading 으로 바꾸고 useOpenRouter.callModel 을 병렬 호출한 뒤, 결과를 dispatch 하고 runMerge 로 통합 모델을 호출한다. 통합이 성공하면 buildHistoryItem 으로 히스토리에 추가되고 effect 가 localStorage 에 기록한다. 컴포넌트 수는 적고 단일 Context 로 충분한 규모이며, 거대 컴포넌트는 없다(최대 163줄).

참고: 이 폴더는 자체 .git 을 가진 중첩 저장소이며, 내부 저장소에는 README.md 한 파일만 커밋되어 있고 나머지(src/, dist/, node_modules/ 포함)는 모두 미추적이다. 상위 platform 저장소에서도 미추적이다. 즉 "dist 커밋" 은 현재 사실이 아니지만, .gitignore 가 없어서 git add . 한 번이면 dist 와 node_modules 가 그대로 커밋되는 상태다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	보안	src/utils/constants.ts:20, src/context/AppContext.tsx:48	VITE_OPENROUTER_API_KEY 를 기본값으로 사용. Vite 는 VITE_ 변수를 번들 JS 에 평문 인라인하므로 .env 가 있는 상태로 빌드하면 키가 dist 에 포함되어 배포 즉시 유출된다. README copy 는 이 사용법을 권장까지 한다
2	높음	설정	프로젝트 루트 (.gitignore 없음)	.gitignore 부재. node_modules/, dist/, 향후 .env 가 커밋 대상이 된다. 1번과 결합 시 키가 저장소 이력에 남는다. 중첩 .git 도 상위 저장소와 충돌
3	높음	버그	src/components/InputSection.tsx:32, :56-58, src/App.tsx:55-69	[다시 생성] 버튼은 disabled 가 없고 Ctrl+Enter 도 isLoading 을 확인하지 않아 실행 중 재실행 가능. 요청 식별자/AbortController 가 없어 늦게 도착한 이전 응답이 새 결과를 덮어쓰고 히스토리가 중복 기록된다
4	중간	버그	src/components/SettingsPanel.tsx:8, :77	draft 를 최초 마운트 시 한 번만 state.settings 로 초기화하고 패널은 닫혀도 언마운트되지 않는다. Header 에서 테마를 바꾼 뒤 설정을 저장하면 이전 테마로 되돌아가고, 취소한 편집 내용도 다음에 열 때 남아 있다
5	중간	버그	src/utils/formatters.ts:6	todayIso 가 toISOString()(UTC) 사용. 한국 시간 00:00~08:59 에는 기본 날짜가 전날로 표시되어 일지 날짜와 파일명이 틀린다
6	중간	버그	src/context/AppContext.tsx:124-134	localStorage.setItem 에 예외 처리 없음. 원문 5000자 + 결과 3개 x 50건 히스토리가 쌓이면 QuotaExceededError 로 effect 에서 예외가 발생해 앱이 깨질 수 있다
7	중간	보안	src/context/AppContext.tsx:125	API Key 를 localStorage 에 평문 저장. 개인 도구로는 허용 가능하나 같은 오리진 XSS 나 공용 PC 에서 노출. 키 삭제 버튼/세션 저장 옵션 없음
8	중간	설정	package.json:10-24	모든 의존성이 "latest". 설치 시점마다 메이저 버전이 바뀌어 재현 불가(현재 Vite 8, TS 6 설치됨). @vitejs/plugin-react 가 dependencies 에 있음

#	심각도	분류	위치(경로:줄)	내용
9	낮음	버그	src/App.tsx:80-82	retryBase 가 클릭 시점 클로저의 state.baseResult1/2 를 사용. 다른 카드가 loading 중이면 통합이 조용히 생략된다. retryFinal(:85-88) 도 실패 시 안내 없이 null 처리
10	낮음	품질	README.md:1, README copy.md	실제 문서가 사본 파일에 있고 README.md 는 제목뿐. 사본은 불필요 파일
11	낮음	품질	src/hooks/useLocalStorage.ts:1-28, src/context/AppContext.tsx:21, :107	useLocalStorage 훅과 RESET_RESULTS 액션 미사용(데드 코드). 저장 로직이 readJson 과 중복
12	낮음	품질	src/components/SettingsPanel.tsx:26-31, src/components/ResultSection.tsx:24	PRD 미충족: 모델 그룹화(optgroup) 없음, 무료 모델 툴팁 없음, 메인 화면 경고 배너 없음, md:grid-cols-2 lg:grid-cols-2 중복으로 데스크톱 레이아웃이 태블릿과 동일
13	낮음	품질	프로젝트 전체	테스트, ESLint 설정 없음. 키워드 추출/파일명 규칙처럼 깨지기 쉬운 순수 함수가 검증되지 않음
14	낮음	설정	src/utils/constants.ts:3-17	모델 ID 하드코딩. OpenRouter 에서 모델이 내려가면 404 에러만 표시되고 갱신 수단이 없다

상세 설명 (심각도 높음)

1번 API 키 번들 노출: DEFAULT_SETTINGS.apiKey 와 getInitialSettings 모두 import.meta.env.VITE_OPENROUTER_API_KEY 를 읽는다. Vite 는 빌드 시 이 값을 문자열 리터럴로 치환하므로, 개발 편의로 .env 에 키를 넣고 npm run build 하면 dist/assets/*.js 를 여는 누구나 키를 얻는다. 현재 dist 에는 placeholder 문자열(sk-or-v1-...)만 있고 실제 키는 확인되지 않았으나, README 가 이 방법을 안내하고 있어 재발 가능성이 높다. env 폴백을 제거하고 키는 설정 패널 입력으로만 받도록 해야 한다.

2번 .gitignore 부재: 현재는 아무것도 커밋되지 않았지만, 내부 저장소의 첫 실질 커밋에서 node_modules(수만 개 파일)와 dist 가 함께 들어갈 가능성이 매우 높다. 1번과 결합하면 키가 git 이력에 영구히 남아 키 폐기가 필요해진다. 또한 중첩 .git 때문에 상위 platform 저장소에 추가할 때 gitlink 로 잡히는 혼란이 생긴다.

3번 동시 실행 경쟁 상태: [정리하기] 버튼만 isLoading 으로 막혀 있고 바로 옆 [다시 생성] 버튼과 Ctrl+Enter 는 막혀 있지 않다. 두 실행이 겹치면 각 실행의 Promise.all 결과가 도착 순서대로 dispatch 되어 카드 내용이 서로 다른 실행에서 섞일 수 있고, 통합 모델이 두 번 호출되어 비용이 두 배가 되며 히스토리도 두 건 기록된다. 유료 모델(Opus 등) 호출이라 비용 문제로 직결된다.

4. 효율성 개선 제안

- 구조/아키텍처
  - App.tsx 의 runAll/runMerge/retryBase/retryFinal 을 usePipeline 커스텀 훅으로 분리하고, 실행마다 runId 를 발급해 최신 실행의 응답만 반영한다. AbortController 로 이전 요청을 취소한다.
  - localStorage 접근을 storage.ts 한 곳으로 모으고(try/catch, 용량 초과 시 오래된 히스토리 정리) 미사용 useLocalStorage 는 삭제한다.
  - 공개 배포가 목표라면 키를 서버리스 프록시(예: Vercel/Netlify Function)로 옮겨야 한다. 개인 로컬 도구로만 쓸 경우 현 구조(사용자 입력 키 + localStorage)는 수용 가능.
- 성능
  - Context 하나에 입력 텍스트까지 있어 타이핑마다 전체 트리가 리렌더되고 draft 를 매 키 입력마다 저장한다. 규모상 체감 문제는 없으나 draft 저장은 300ms 디바운스가 적절하다.
  - 히스토리에 기본 모델 결과까지 전부 저장하므로 50건 상한 외에 총 바이트 상한을 둔다.
- 개발 생산성

    - 의존성 버전 고정(현재 설치본 기준 `react@19.2.6`, `vite@8.0.14`, `tailwindcss@4.3.0`, `typescript@6.0.3`), `package-lock.json` 커밋.
    - Vitest 로 `formatters.ts`(extractKeywords, removeKeywordBlock, getDateParts, makeFinalFilename)와 `mapError` 단위 테스트 추가. ESLint(react-hooks 규칙) 추가 시 4번 같은 상태 동기화 버그를 조기에 잡을 수 있다.
- 운영/배포
    - `.gitignore`(node_modules, dist, .env, .env.local, `!.env.example`) 추가, 중첩 저장소를 유지할지 상위 저장소로 합칠지 결정.
    - README 를 `README copy.md` 내용으로 교체하고 env 키 안내 문단은 삭제.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	루트에 `.gitignore` 추가(node_modules, dist, .env, .env.*, `!.env.example`), 중첩 `.git` 처리 방침 결정	P0	S	Sonnet 5.5	설정 파일 작업
2	constants.ts:20, AppContext.tsx:48 의 VITE_OPENROUTER_API_KEY 폴백 제거, .env.example 과 README 의 env 키 안내 삭제, 설정 패널에 "키 삭제" 버튼 추가	P0	S	Opus 5.5	범위는 작지만 키 유출 관련이라 누락 시 비용이 큼
3	동시 실행 방지: [다시 생성] 버튼 `disabled={isLoading}`, Ctrl+Enter 에 isLoading 가드, 실행별 runId 로 오래된 응답 무시 및 AbortController 취소	P0	M	Opus 5.5	비동기 경쟁 상태와 유료 API 중복 호출 비용
4	SettingsPanel 이 열릴 때마다 draft 를 state.settings 로 재설정(useEffect([state.isSettingsOpen]) 또는 열릴 때만 마운트)	P0	S	Sonnet 5.5	단순 상태 동기화 버그
5	`todayIso` 를 로컬 날짜 기준(getFullYear/getMonth/getDate)으로 수정	P1	S	Sonnet 5.5	단순 버그
6	localStorage 쓰기를 try/catch 로 감싸고 QuotaExceeded 시 오래된 히스토리부터 삭제 후 재시도, draft 저장 300ms 디바운스	P1	S	Sonnet 5.5	단일 파일 방어 코드
7	`package.json` 의 `latest` 를 현재 설치 버전으로 고정, `@vitejs/plugin-react` 를 devDependencies 로 이동	P1	S	Sonnet 5.5	설정 정리
8	`README copy.md` 내용을 `README.md` 로 옮기고 사본 삭제, 보안 주의사항 갱신	P1	S	Sonnet 5.5	문서화
9	`retryBase/retryFinal` 이 최신 상태를 쓰도록 수정하고 통합 생략 시 사유 토스트 표시	P1	S	Opus 5.5	3번 파이프라인 수정과 같은 코드 경로라 함께 처리
10	미사용 src/hooks/useLocalStorage.ts 와 RESET_RESULTS 액션 삭제	P2	S	Sonnet 5.5	데드 코드 정리
11	PRD 미구현 UI: 모델 드롭다운 optgroup(유료/무료 또는 공급사별), 무료 모델 툴팁, 메인 화면 상단 노란 경고 배너, `lg` 레이아웃 정리	P2	M	Sonnet 5.5	반복적 UI 작업
12	Vitest 도입 후 formatters.ts 4개 함수와 `mapError` 단위 테스트 작성	P2	M	Opus 5.5	테스트 설계
13	ESLint(typescript-eslint, react-hooks) 설정 및 `lint` 스크립트 추가	P2	S	Sonnet 5.5	설정 파일 작업
14	(공개 배포 시에만) OpenRouter 호출을 서버리스 프록시로 이전하고 사용량 제한 추가	P2	L	Fable 5.1	인증/키 관리/비용 보호 설계

6. 종합 평가

- 강점

1. PRD 를 충실히 따른 깔끔한 구조. 파일당 200줄 미만, 타입 정의가 명확하고 strict 타입 체크가 통과한다.
2. 카드 단위 독립 에러 처리와 부분 실패 허용(한쪽 실패 시 통합 생략)이 명세대로 구현되어 있다.
3. 의존성이 최소(React, lucide, Tailwind)이고 백엔드가 없어 유지 비용이 거의 없다.

- 약점
  1. env 기반 API 키 번들 주입 경로와 `.gitignore` 부재로 키 유출 위험이 열려 있다.
  2. 실행 중 재실행이 막혀 있지 않아 결과 혼선과 유료 모델 중복 과금이 가능하다.
  3. 버전 미고정, 테스트/린트 부재, README 사본 등 저장소 위생이 정리되지 않았다(내부 git 에 README 한 파일만 커밋).
- 모델별 비율 (14개 작업 기준): Fable 5.1 7% (1개) / Opus 5.5 29% (4개) / Sonnet 5.5 64% (9개)
- 권장 진행 순서
  - 1단계: 1, 2, 3, 4 (보안 위생과 경쟁 상태 차단, 반나절)
  - 2단계: 5, 6, 7, 8, 9 (안정성과 재현성, 저장소 첫 정식 커밋)
  - 3단계: 10, 11, 12, 13 (PRD 잔여 UI, 테스트/린트). 14는 공개 배포 결정 시에만
- 권고: 계속 발전. 완성도가 약 85%이고 남은 작업 대부분이 S 규모라 1~2일이면 PRD 완료 기준을 충족한다. 아카이브하기엔 실사용 가치가 분명하고, 템플릿 통합은 다른 프로젝트에서 `useOpenRouter` + "2모델 병렬 후 통합" 패턴이 재사용될 때 추출하면 충분하다.
- 주 담당 모델: Sonnet 5.5. 코드베이스가 1,100줄 남짓으로 작고 구조가 단순하며, 남은 작업의 대부분이 설정 파일, 단순 버그, 문서, 반복 UI 정리다. 다만 2, 3, 9번(키 처리와 비동기 파이프라인)은 실수 시 키 유출과 유료 API 중복 과금으로 이어지므로 해당 작업만 Opus 5.5 에 맡기고, 서버 프록시가 필요해지는 시점에만 Fable 5.1 을 투입하는 구성이 가장 비용 효율적이다.